

Pet Territory Wars — MVP Database Entity Review & ER Diagram v1.0

Veritabanı: PostgreSQL + PostGIS

H3 veri tipi: BIGINT

Kapsam: Faz 0 Backend Simulation + Faz 1 Barebone Mobile

1. Amaç

Bu doküman MVP veritabanındaki entity'leri, kullanım amaçlarını ve aralarındaki ilişkileri tanımlar.

Temel hedefler:

- Gereksiz tablo ve veri tekrarını önlemek
 - Authoritative veriyi açıkça belirlemek
 - Walk işlemlerini idempotent yapmak
 - Aynı anda binlerce kullanıcının yürüyüş göndermesini desteklemek
 - Aynı hex üzerindeki eşzamanlı değişiklikleri güvenli yönetmek
 - Engine sonuçlarını açıklanabilir ve denetlenebilir tutmak
-

2. MVP Entity Listesi

MVP için önerilen entity'ler:

1. players
 2. pets
 3. cities
 4. city_boundaries
 5. rule_sets
 6. seasons
 7. walks
 8. walk_points
 9. walk_processing_jobs
 10. territory_hexes
 11. walk_hex_results
 12. territory_change_log
 13. player_season_scores
 14. player_lifetime_stats
 15. score_change_log
 16. outbox_events
-

3. Core Entity'ler

3.1 players

Oyuncunun oyun içi kimliğini ve temel profilini tutar.

Amaç:

- Oyuncuyu sistem içinde benzersiz tanımlamak
- Authentication hesabını oyun profilinden ayırmak
- Oyuncunun aktif, askıya alınmış veya silinmiş durumunu tutmak
- Ana şehir bilgisini saklamak

Temel kolonlar:

```
id
auth_subject
display_name
home_city_id
status
created_at
updated_at
deleted_at
```

Source of truth: Oyuncu profili

İlişkiler:

```
Player 1 — N Pet
Player 1 — N Walk
Player 1 — N TerritoryHex
Player 1 — N PlayerSeasonScore
Player 1 — 1 PlayerLifetimeStats
```

Player içine hex listesi, yürüyüş listesi veya skor geçmişi gömülmez.

3.2 pets

Oyuncunun yürüyüş yaptığı evcil hayvanı temsil eder.

Amaç:

- Walk ile pet ilişkisini kurmak
- Pet profilini Player tablosundan ayırmak
- İleride çoklu pet ve breed sistemi için genişleme alanı bırakmak

Temel kolonlar:

```
id
owner_id
name
status
created_at
updated_at
deleted_at
```

Source of truth: Pet profili

MVP'de pet bonusu, level veya evolution bilgisi bulunmaz.

3.3 cities

Oyunun aktif olduğu şehirleri tanımlar.

Amaç:

- City Activation Model'i yönetmek
- Walk ve territory verilerini şehir bazında ayırmak
- Şehir timezone bilgisini saklamak
- Şehirde kullanılacak RuleSet'i belirlemek

Temel kolonlar:

```
id
code
name
country_code
timezone
status
active_rule_set_id
created_at
updated_at
```

Durumlar:

```
ACTIVE
COMING_SOON
FROZEN
```

Source of truth: Şehir durumu

City içine şehirdeki bütün hex veya oyuncular gömülmez.

3.4 city_boundaries

Şehirlerin idari sınır polygon'larını tutar.

Amaç:

- Walk noktalarının aktif şehir içinde olup olmadığını kontrol etmek
- Şehir sınırlarını versiyonlamak
- PostGIS ile point-in-polygon sorgusu yapmak

Temel kolonlar:

```
id
city_id
version
boundary
is_active
source
created_at
```

`boundary` tipi:

```
geometry(MultiPolygon, 4326)
```

Source of truth: Şehir geometrisi

Bir şehirde yalnızca bir boundary sürümü aktif olabilir.

3.5 rule_sets

Game Engine'in kullandığı versiyonlanmış kuralları tutar.

Amaç:

- Calculator Rules parametrelerini saklamak
- Eski yürüyüşlerin hangi kurallarla hesaplandığını açıklamak
- Kuralları engine kodundan bağımsız değiştirmek

Temel kolonlar:

```
id
version
status
rules
checksum
```

```
created_at
activated_at
retired_at
```

rules tipi:

JSONB

Source of truth: Engine parametreleri

Aktif olarak kullanılmış bir RuleSet değiştirilmez. Yeni değerler için yeni sürüm oluşturulur.

3.6 seasons

Haftalık oyun dönemlerini temsil eder.

Amaç:

- Haftalık skorları ayrı dönemlere bağlamak
- Sezon başlangıç ve bitiş zamanlarını tutmak
- Sezonda kullanılan RuleSet'i kaydetmek
- Sezon geçmişini korumak

Temel kolonlar:

```
id
season_number
name
status
starts_at
ends_at
rule_set_id
created_at
closed_at
```

Durumlar:

```
SCHEDULED
ACTIVE
CLOSED
ARCHIVED
```

Source of truth: Sezon dönemi

Season içine oyuncu skorları gömülmez.

4. Walk Entity'leri

4.1 walks

Bir oyuncunun tek yürüyüş oturumunu temsil eder.

Amaç:

- Walk lifecycle'ını yönetmek
- Yürüyüşün bir kez işlenmesini sağlamak
- Engine ve RuleSet sürümünü kaydetmek
- Validation ve hesaplama sonuç özetini tutmak

Temel kolonlar:

```
id
player_id
pet_id
city_id
season_id
rule_set_id

status
validation_status

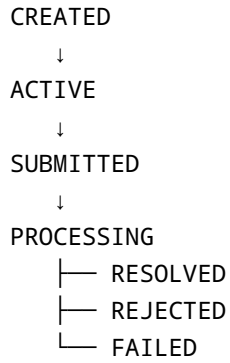
started_at
ended_at
submitted_at
processing_started_at
evaluated_at
resolved_at

total_distance_m
valid_distance_m
duration_seconds
valid_route_ratio

engine_version
input_hash
idempotency_key
rejection_reasons
processing_attempts

created_at
updated_at
```

Durum akışı:



Source of truth: Walk lifecycle

Aynı oyuncu ve aynı idempotency key ile ikinci Walk sonucu oluşturulamaz.

4.2 walk_points

Flutter'dan gelen raw GPS noktalarını tutar.

Amaç:

- Engine'e işlenecek rota verisini sağlamak
- GPS validation ve anti-spoofing analizi yapmak
- Kısa süreli debug ve kullanıcı itirazlarını incelemek

Temel kolonlar:

```
walk_id
sequence_no
recorded_at
location
accuracy_m
altitude_m
speed_mps
is_mock_location
received_at
```

Primary key:

```
walk_id + sequence_no
```

`location` tipi:

```
geography(Point, 4326)
```

Source of truth: Raw walk route

Ayrıca `latitude` ve `longitude` kolonları tutulmaz. Aynı konum iki farklı biçimde kopyalanmaz.

Raw GPS verisi sınırsız saklanmaz.

Başlangıç retention kararı:

60 gün

4.3 walk_processing_jobs

Asenkron Walk işleme kuyruğunu temsil eder.

Amaç:

- Walk resolve işlemini HTTP isteğinden ayırmak
- Worker'ların kontrollü biçimde iş almasını sağlamak
- Retry ve hata yönetimi yapmak
- Aynı Walk'ın iki worker tarafından aynı anda işlenmesini engellemek

Temel kolonlar:

```
id
walk_id
status
priority
attempt_count
available_at
locked_at
locked_by
last_error
created_at
completed_at
```

Durumlar:

```
PENDING
PROCESSING
COMPLETED
RETRY
DEAD
```

`walk_id` unique olmalıdır.

Source of truth: Walk processing durumu

Walk lifecycle ile job lifecycle aynı kavram değildir. Bu nedenle ayrı entity olarak tutulur.

5. Territory Entity'leri

5.1 territory_hexes

Oyunun aktif sahiplik ve dominance state'ini tutan en kritik tablodur.

Amaç:

- Hex sahibini saklamak
- Stored dominance değerini tutmak
- Concurrent saldırıları version ile kontrol etmek
- Harita ve ownership sorgularının authoritative kaynağı olmak

Temel kolonlar:

```
city_id  
hex_id  
owner_id  
dominance  
last_updated_at  
last_visited_at  
version
```

Primary key:

```
city_id + hex_id
```

hex_id:

```
BIGINT
```

Source of truth:

```
Hex owner  
Stored dominance  
Hex version
```

Boş bütün dünya hex'leri önceden oluşturulmaz. Bir hex ilk kez etkilendiğinde kayıt oluşturulur.

Hex polygon'u tabloda tutulmaz. H3 ID'den üretilebilir.

5.2 walk_hex_results

Engine'in bir yürüyüş için hesapladığı hex bazlı sonuçları tutar.

Amaç:

- Bir yürüyüşün hangi hex'leri etkilediğini göstermek
- Engine sonucunu açıklamak
- Walk replay ve debug desteği sağlamak
- Kullanıcıya yürüyüş sonuç özeti sunmak

Temel kolonlar:

```
walk_id  
hex_id  
presence_seconds  
distance_m  
entry_count  
action_type  
dominance_before  
dominance_after  
previous_owner_id  
new_owner_id  
created_at
```

Primary key:

```
walk_id + hex_id
```

Action türleri:

```
NO_CHANGE  
EMPTY_CAPTURE  
OWNER_DEFENSE  
ENEMY_ATTACK  
OWNERSHIP_TRANSFER
```

Source of truth: Engine'in Walk için ürettiği hesap sonucu

Mevcut territory state'inin kaynağı değildir.

5.3 territory_change_log

Database'e gerçekten uygulanmış territory değişikliklerini tutar.

Amaç:

- Ownership ve dominance değişikliklerini audit etmek
- Retry veya conflict sonrası gerçekten uygulanan sonucu kaydetmek
- Skor reconciliation yapmak
- Hile, hata ve kullanıcı itirazlarını incelemek

Temel kolonlar:

```
id
walk_id
city_id
hex_id
change_type
previous_owner_id
new_owner_id
dominance_before
dominance_after
previous_version
new_version
occurred_at
created_at
```

Source of truth: Territory değişiklik geçmişi

Fark:

```
walk_hex_results
→ Engine ne hesapladı?

territory_change_log
→ Database'e ne uygulandı?
```

6. Score Entity'leri

6.1 player_season_scores

Oyuncunun belirli bir sezon ve şehirdeki haftalık skorlarını tutar.

Amaç:

- Leaderboard sorgularını hızlandırmak
- Her istekte bütün territory kayıtlarını tekrar saymamak
- Haftalık capture, steal ve defense değerlerini tutmak

Temel kolonlar:

```
player_id
season_id
city_id
active_hex_count
capture_count
steal_count
defense_count
score_reached_at
version
updated_at
```

Primary key:

```
player_id + season_id + city_id
```

Source of truth durumu:

Territory ownership için source of truth değildir.
Leaderboard için güncel derived state'tir.

`active_hex_count`, `territory_hexes` üzerinden yeniden hesaplanabilir.

6.2 player_lifetime_stats

Oyuncunun sezonlardan bağımsız kalıcı istatistiklerini tutar.

Amaç:

- Lifetime capture ve steal değerlerini saklamak
- Toplam geçerli yürüyüş ve mesafeyi tutmak
- Haftalık skorlarla kalıcı skorları ayırmak

Temel kolonlar:

```
player_id
capture_count
steal_count
valid_walk_count
total_distance_m
alpha_win_count
version
updated_at
```

Primary key:

```
player_id
```

Source of truth: Lifetime oyuncu istatistikleri

Lifetime değerler her sezon satırında tekrar edilmez.

6.3 score_change_log

Engine tarafından üretilen ve database'e uygulanan skor hareketlerini tutar.

Amaç:

- Skor değişikliklerinin nedenini açıklamak
- Aynı Walk skorunun iki kez uygulanmasını engellemek
- Skor sayaçlarını gerektiğinde yeniden oluşturmak
- Audit ve reconciliation yapmak

Temel kolonlar:

```
id  
walk_id  
player_id  
season_id  
city_id  
metric  
delta  
reason  
created_at
```

Unique constraint:

```
walk_id + player_id + metric + reason
```

Metric örnekleri:

```
ACTIVE_HEX  
CAPTURE  
STEAL  
DEFENSE  
LIFETIME_CAPTURE  
LIFETIME_STEAL  
VALID_WALK  
WALK_DISTANCE
```

Source of truth: Skor değişiklik geçmişi

`processed_score_changes` isimli ayrı tablo kullanılmaz. Idempotency ve audit aynı entity ile çözülür.

7. Event Entity

7.1 outbox_events

Ana transaction ile üretilen domain event'lerini güvenli biçimde saklar.

Amaç:

- Database commit sonrası event kaybını önlemek
- Push, analytics ve cache işlemlerini Walk transaction'ından ayırmak
- Event'lerin retry ile işlenmesini sağlamak

Temel kolonlar:

```
id
event_type
aggregate_type
aggregate_id
payload
occurred_at
created_at
available_at
processed_at
attempt_count
last_error
locked_at
locked_by
```

`payload` tipi:

```
JSONB
```

Source of truth: Event delivery durumu

Outbox şu işleri doğrudan yapmaz:

```
Push notification
WebSocket
Analytics
Cache invalidation
```

Bunlar worker'lar tarafından daha sonra yapılır.

8. ER Diagram

```
erDiagram
    PLAYERS {
        uuid id PK
        string auth_subject UK
        string display_name
        uuid home_city_id FK
        string status
    }

    PETS {
        uuid id PK
        uuid owner_id FK
        string name
        string status
    }

    CITIES {
        uuid id PK
        string code UK
        string name
        string country_code
        string timezone
        string status
        uuid active_rule_set_id FK
    }

    CITY_BOUNDARIES {
        uuid id PK
        uuid city_id FK
        int version
        multipolygon boundary
        boolean is_active
    }

    RULE_SETS {
        uuid id PK
        string version UK
        string status
        jsonb rules
        string checksum UK
    }

    SEASONS {
        uuid id PK
        int season_number UK
        string status
        timestamp starts_at
        timestamp ends_at
    }
```

```

        uuid rule_set_id FK
    }

    WALKS {
        uuid id PK
        uuid player_id FK
        uuid pet_id FK
        uuid city_id FK
        uuid season_id FK
        uuid rule_set_id FK
        string status
        string input_hash
        string idempotency_key
    }

    WALK_POINTS {
        uuid walk_id PK, FK
        int sequence_no PK
        timestamp recorded_at
        geography location
        float accuracy_m
        boolean is_mock_location
    }

    WALK_PROCESSING_JOBS {
        uuid id PK
        uuid walk_id FK, UK
        string status
        int attempt_count
        timestamp available_at
    }

    TERRITORY_HEXES {
        uuid city_id PK, FK
        bigint hex_id PK
        uuid owner_id FK
        int dominance
        bigint version
        timestamp last_updated_at
    }

    WALK_HEX_RESULTS {
        uuid walk_id PK, FK
        bigint hex_id PK
        string action_type
        int dominance_before
        int dominance_after
        uuid previous_owner_id
        uuid new_owner_id
    }

```



```

TERRITORY_CHANGE_LOG {
    uuid id PK
    uuid walk_id FK
    uuid city_id FK
    bigint hex_id
    string change_type
    bigint previous_version
    bigint new_version
}

PLAYER_SEASON_SCORES {
    uuid player_id PK, FK
    uuid season_id PK, FK
    uuid city_id PK, FK
    int active_hex_count
    int capture_count
    int steal_count
    int defense_count
}

PLAYER_LIFETIME_STATS {
    uuid player_id PK, FK
    bigint capture_count
    bigint steal_count
    bigint valid_walk_count
    bigint total_distance_m
}

SCORE_CHANGE_LOG {
    uuid id PK
    uuid walk_id FK
    uuid player_id FK
    uuid season_id FK
    uuid city_id FK
    string metric
    bigint delta
    string reason
}

OUTBOX_EVENTS {
    uuid id PK
    string event_type
    string aggregate_type
    string aggregate_id
    jsonb payload
    timestamp processed_at
}

CITIES ||--o{ PLAYERS : home_city
PLAYERS ||--o{ PETS : owns
CITIES ||--o{ CITY_BOUNDARIES : has

```

```

RULE_SETS ||--o{ CITIES : active_rules
RULE_SETS ||--o{ SEASONS : used_by

PLAYERS ||--o{ WALKS : performs
PETS ||--o{ WALKS : accompanies
CITIES ||--o{ WALKS : contains
SEASONS ||--o{ WALKS : includes
RULE_SETS ||--o{ WALKS : calculated_with

WALKS ||--o{ WALK_POINTS : contains
WALKS ||--o{ WALK_PROCESSING_JOBS : processed_by
WALKS ||--o{ WALK_HEX_RESULTS : produces
WALKS ||--o{ TERRITORY_CHANGE_LOG : applies
WALKS ||--o{ SCORE_CHANGE_LOG : produces

CITIES ||--o{ TERRITORY_HEXES : contains
PLAYERS ||--o{ TERRITORY_HEXES : owns

PLAYERS ||--o{ PLAYER_SEASON_SCORES : has
SEASONS ||--o{ PLAYER_SEASON_SCORES : groups
CITIES ||--o{ PLAYER_SEASON_SCORES : ranks

PLAYERS ||--|| PLAYER_LIFETIME_STATS : has

PLAYERS ||--o{ SCORE_CHANGE_LOG : receives
SEASONS ||--o{ SCORE_CHANGE_LOG : groups
CITIES ||--o{ SCORE_CHANGE_LOG : scopes

```

9. Source of Truth Özeti

Veri	Authoritative Entity
Oyuncu profili	players
Pet profili	pets
Şehir durumu	cities
Şehir polygon'u	city_boundaries
Engine kuralları	rule_sets
Sezon zamanı	seasons
Walk lifecycle	walks
Raw rota	walk_points
Job durumu	walk_processing_jobs
Hex sahibi	territory_hexes

Veri	Authoritative Entity
Stored dominance	territory_hexes
Engine Walk sonucu	walk_hex_results
Uygulanan territory geçmişı	territory_change_log
Haftalık skor state'i	player_season_scores
Lifetime skor state'i	player_lifetime_stats
Skor değışiklik geçmişı	score_change_log
Event delivery	outbox_events

10. Transaction İlişkisi

Bir Walk başarıyla işlendiğinde aynı kısa transaction içinde:

```
territory_hexes
territory_change_log
player_season_scores
player_lifetime_stats
score_change_log
walk_hex_results
walks
walk_processing_jobs
outbox_events
```

güncellenir.

Engine hesabı transaction dışında çalışır.

Transaction yalnızca sonuçları uygulamak için açılır.

Hex güncellemeleri:

```
hex_id ASC
```

sırasıyla uygulanır.

Bu deadlock riskini azaltır.

11. MVP Dışında Bırakılan Entity'ler

Şimdilik eklenmeyecek tablolar:

```
guilds
guild_members
professional_walker_profiles
breed_classes
inventories
wallets
boosters
sponsor_territories
npc_players
events
badges
achievements
notifications
chat
social_feed
```

Push notification ve WebSocket daha sonra eklenecektir.

Outbox altyapısı gelecekte bunları destekleyebilir.

12. Nihai MVP Kararı

MVP veritabanı modeli 16 entity'den oluşur.

En kritik authoritative entity'ler:

```
walks
walk_points
territory_hexes
player_season_scores
player_lifetime_stats
rule_sets
```

Güvenilirlik entity'leri:

```
walk_processing_jobs
walk_hex_results
territory_change_log
```

score_change_log
outbox_events

Temel veri tasarımı kararları:

H3 ID PostgreSQL'de BIGINT tutulur.
Territory polygon'u kopyalanmaz.
Raw GPS yalnızca walk_points içinde tutulur.
Player içinde yürüyüş veya hex listesi tutulmaz.
Weekly ve lifetime skorlar ayrıdır.
Walk ve job lifecycle ayrıdır.
Engine sonucu ile uygulanan DB sonucu ayrıdır.
Skor audit ve idempotency aynı score_change_log ile çözülür.
Hex state optimistic version ile korunur.